

Relazione - Versione 2: Executor

Obiettivo della Versione 2

La Versione 2 introduce un nuovo componente: l'Executor.

Nella Versione 1 l'agente eseguiva direttamente i tool. Questo funzionava, ma non era una buona separazione delle responsabilità.

L'obiettivo della Versione 2 è spostare la responsabilità dell'esecuzione fuori dall'Agent.

La nuova regola diventa:

l'Agent coordina, l'Executor esegue.

Cosa cambia rispetto alla Versione 1

Nella Versione 1 il flusso era:

```
User request
↓
Router
↓
Parser
↓
Agent executes tool
↓
Memory
```

Nella Versione 2 diventa:

```
User request
↓
Router
↓
Parser
↓
Executor
↓
Agent stores state in memory
```

La differenza principale è questa:

```
prima: Agent esegue direttamente
ora: Agent delega all'Executor
```

Questo rende il codice più ordinato e più vicino a una struttura professionale.

Nuovo componente: Executor

L'Executor ha due responsabilità principali:

1. 2.

validare gli argomenti; eseguire il tool corretto.

Per questo nella sezione Executor troviamo:

```
arguments_are_valid(arguments)
```

e:

```
executor(tool_name, arguments)
```

La funzione `arguments_are_valid()` controlla che gli argomenti esistano e che non siano `None`.

La funzione `executor()` riceve il nome del tool e gli argomenti già estratti, poi esegue la funzione corretta.

Ruolo aggiornato dell'Agent

L'Agent non esegue più direttamente il tool.

Ora fa solo questo:

1. 2. 3. 4. 5.

riceve la richiesta; usa il Router per scegliere il tool; usa il Parser per estrarre gli argomenti; passa tutto all'Executor; salva lo stato in memoria.

Quindi l'Agent diventa più pulito.

Il suo ruolo è coordinare il flusso, non gestire i dettagli dell'esecuzione.

Flusso completo della Versione 2

Esempio:

```
What is 2 + 3?
```

Il Router sceglie:

```
"addition"
```

Il Parser estrae:

```
{"a": 2, "b": 3}
```

L'Executor esegue:

```
addition(a=2, b=3)
```

Il risultato è:

```
5
```

Lo stato finale diventa:

```
{
  "request": "What is 2 + 3?",
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

Struttura corretta del notebook

La Versione 2 deve avere questa sequenza:

1. Imports
2. Tools
3. Parser
4. Router
5. Executor
6. Agent

7. Tests

Notes

Questa struttura è importante perché riflette il flusso logico dell'agente.

In particolare, `arguments_are_valid()` deve stare nella sezione `Executor`, non più nella sezione `Agent`.

Questo perché la validazione degli argomenti fa parte dell'esecuzione.

Cosa abbiamo imparato nella Versione 2

La Versione 2 insegna un concetto molto importante:

un agente professionale non deve fare tutto da solo.

Ogni componente deve avere una responsabilità chiara.

In questa versione abbiamo separato:

```
Agent = coordinamento  
Executor = validazione + esecuzione
```

Questa è una scelta architetturale importante.

Anche se il codice è ancora semplice, la struttura inizia a diventare più seria.

Test principali

I test verificano che:

```
agent("What is 2 + 3?")
```

produca risultato 5 .

Verificano anche che:

```
agent("Hello, what is 2 + 3?")
```

venga interpretato come una richiesta di somma, non come semplice saluto.

Infine verificano che:

```
agent("Hello, my name is luca.")
```

produca:

```
"Hello Luca, nice to meet you!"
```

Questi test controllano `Router`, `Parser`, `Executor` e `Agent` insieme.

Limiti della Versione 2

La Versione 2 è più ordinata della Versione 1, ma ha ancora alcuni limiti:

il parser è ancora rigido; usa condizioni specifiche per i parametri; la memoria è ancora una semplice lista; non esiste ancora un planner; non c'è ancora una LLM; i tool sono ancora registrati manualmente.

Questi limiti verranno affrontati nelle versioni successive.

Conclusione

La Versione 2 rappresenta il primo vero passo verso una struttura professionale.

Abbiamo separato l'esecuzione dal coordinamento.

L'Agent non esegue più direttamente i tool. L'Executor riceve il tool scelto e gli argomenti, li valida ed esegue la funzione.

Il progetto ora è più pulito, più leggibile e più facile da estendere.

Il prossimo passo sarà la Versione 3, dove miglioreremo il Parser rendendolo generico.